

Phase 2 Recap & Mini System

Phase 2 · Intermediate · Estimated time: 2 hours

1. Concept From First Principles: Putting It All Together

If any chapter in this Phase felt shaky on this recap, treat that as useful signal, not failure — go back to that specific chapter before Phase 3, because Phase 3 (Advanced) assumes every Phase 2 mental model — replication, sharding, consistent hashing, consistency models, message queues, idempotency, rate limiting, microservices, API gateways, and search — is already load-bearing, the way Phase 1's recap assumed Phase 1's fundamentals.

This chapter has no new concepts. Its job is to force you to combine most of Phase 2 into one coherent, interview-realistic design. We'll design the backend for a **ride-booking notification system**: every time a driver is assigned, arrives, or a trip status changes, the system must reliably notify the rider (and driver) via push notification and SMS, at the scale of a Careem- or Swiggy-sized platform.

Step 1 — Clarify requirements

- Functional: when a trip event occurs (driver assigned, driver arrived, trip started, trip completed), send a push notification and, as a fallback, an SMS if push fails or isn't available.
- Functional: never send the same notification for the same event twice, even if the triggering event is retried or delivered more than once.
- Non-functional: the notification-sending path must not slow down or risk breaking the core trip-matching flow that triggers it.
- Non-functional: SMS sending must respect the SMS provider's hard rate limit of 200 requests/second.
- Non-functional: the system must survive a single service instance or database shard failing.

Step 2 — Back-of-the-envelope estimation

Assume 2 million trips/day, averaging 4 notification-worthy events per trip (assigned, arrived, started, completed) — roughly 8 million notification events/day, or about 93/second on average, with peak traffic (lunch/dinner rush) reasonably assumed at 5x average, so design for roughly 500 events/second at peak. If 15% of push notifications fail or aren't deliverable and fall back to SMS, that's about 1.2 million SMS/day, or roughly 14/second average and up to 70/second at peak — comfortably under the assumed 200/second SMS provider limit, but close enough that a rate limiter is clearly still warranted as a safety net, not an afterthought.

Step 3 — High-level architecture

Trip service (part of the core ride-booking microservices, Chapter 11) publishes a **trip.event** message onto a Kafka-style topic (Chapter 8) whenever a notifiable event occurs, rather than calling the notification service synchronously — this directly satisfies the requirement that notification sending must never slow down or risk the core trip flow. A dedicated **Notification Service** (behind the platform's API gateway, Chapter 12, for any external-facing endpoints it exposes) consumes from this topic, and is itself horizontally scaled and sharded (Chapter 6) by `user_id` so that each shard handles a

predictable slice of users.

Step 4 — The idempotency and rate-limiting deep dive

Because the trip.event topic delivers at-least-once (Chapter 8), the same event can arrive at the Notification Service more than once. Each event carries a stable idempotency key (trip_id + event_type, Chapter 9), and the service checks a fast key-value store (Redis) for that key before sending; if already sent, it skips silently. Before dispatching any SMS, the service checks a token-bucket rate limiter (Chapter 10) backed by a shared Redis counter, capped safely under the provider's 200/second limit, queuing (not dropping) any SMS that would exceed the instantaneous rate, since notifications a few seconds late are far preferable to notifications silently lost.

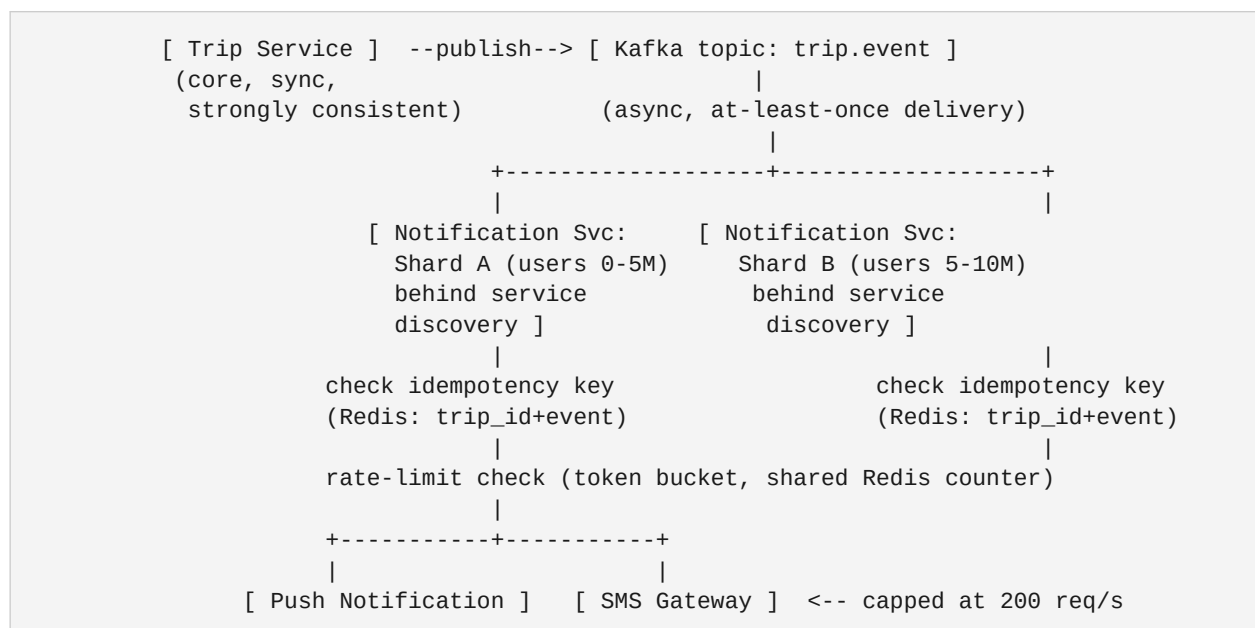
Step 5 — Sanity-check against non-functional requirements

Availability: the Notification Service runs multiple instances behind service discovery (Chapter 12), so losing one instance doesn't lose events, since Kafka retains unconsumed messages (Chapter 8) until a healthy consumer picks them up. **Consistency:** notification delivery is intentionally eventually consistent (Chapter 3) — a few seconds' delay between a trip event and its notification is acceptable, unlike the strong consistency the trip/payment core itself requires. **Sharding:** each shard's Redis-backed idempotency and rate-limit state is scoped correctly so shards don't need to coordinate with each other for these checks, keeping the design horizontally scalable without cross-shard chatter.

2. Real-World Example / Case Study

This shape — event-driven, async, idempotent, rate-limited notification dispatch decoupled from the core transactional flow — is exactly how Swiggy's order-status notifications, Razorpay's payment-confirmation SMS/webhooks, and Careem's trip-status pushes are built in practice, precisely because none of these platforms can afford their core order/payment/trip logic to be slowed down, or worse, blocked, by a flaky downstream SMS or push-notification provider.

3. Diagram



4. Trade-offs Table

Decision made	Alternative considered	Why this choice, at this scale
Async event via Kafka, not sync call	Notification Service called directly by Trip Service	Decouples a flaky notification path from the core trip flow
Idempotency key per trip_id+event_type	No dedup, rely on 'exactly-once' infra	At-least-once delivery is the realistic default; dedup is cheap and reliable
Token-bucket rate limiter on SMS path only	Rate-limit push notifications too	Push has no hard external cap; SMS provider does — limit only where needed
Shard Notification Service by user_id	Single monolithic notification service	Matches Phase 2 Chapter 6's scaling pattern; avoids one service being a bottleneck
Eventual consistency for notification delivery	Strong consistency (block trip update until notified)	A few seconds' delay is fine; blocking the trip flow on notification delivery is not

5. Common Interview Questions

Q1: "Walk me through your architecture for this system, end to end."

Model approach: Follow the same order used in this chapter — requirements, estimation, architecture, then the idempotency/rate-limiting deep dive, narrating the reasoning behind each decision as you go, exactly as a real interview expects.

Q2: "What happens if the SMS provider is completely down for 10 minutes?"

Model approach: Because delivery is async via Kafka/a queue, undelivered SMS events simply remain unacknowledged and get retried by the consumer once the provider recovers, rather than being lost — this is exactly the load-leveling and durability benefit Chapter 8 described, and it's why the design chose async dispatch over a synchronous call in the first place.

Q3: "What's the very first thing that breaks if trip volume grows 10x overnight?"

Model approach: Reason from the estimation numbers — at 10x, peak SMS volume would approach roughly 700/second, exceeding the assumed 200/second provider cap entirely, meaning the rate limiter's queue would grow unboundedly and notifications would arrive increasingly late; the real fix at that scale is negotiating a higher provider limit or adding a second SMS provider, not a purely architectural change.

6. Practice Exercises (Capstone)

- 1 Without looking back, design this notification system from scratch: requirements, estimation, architecture diagram, and identify which two Phase 2 chapters you leaned on most.
- 2 Add a new requirement: notifications must be delivered in the correct order per trip (e.g., 'arrived' must never be shown before 'assigned'). Explain what changes, referencing Chapter 3's causal consistency.
- 3 Using your own estimation numbers for a Talabat- or Noon-scale platform, identify at what point this design would need to add a second SMS provider, and justify the trigger.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Complete System Design Roadmap for 90 Days FAANG Interviews"	Good sanity check of how Phase 2 topics fit the broader interview roadmap.
English	"System Design Verticals — How Big Tech Engineers Think About Architecture"	Reinforces end-to-end architectural reasoning across the topics this Phase covered.
Hindi	"Complete System Design Roadmap 2025 HLD & LLD" — CodeHelp	Full Hindi roadmap, worth revisiting now that Phase 2 is complete.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Comprehensive bilingual review covering every Phase 2 topic in one course.

8. Key Takeaways

- A repeatable order works for any HLD problem: requirements -> estimation -> architecture -> deep dive -> trade-offs.
- Async, event-driven design (message queues) is how you decouple a flaky or slow dependency from a critical core flow.
- Idempotency keys and rate limiters are cheap, standard safeguards that belong in almost every real dispatch pipeline.
- Sharding a service by a stable key (like `user_id`) lets it scale horizontally without cross-shard coordination.
- Not every part of a system needs the same consistency guarantee — notification delivery can be eventually consistent even when the core trip/payment flow is not.
- Estimation numbers should directly justify architecture decisions, including when a design will need to change (like adding a second SMS provider).
- You are now ready for Phase 3, which layers advanced reliability, security, and distributed-systems patterns on top of everything Phase 1 and 2 built.